

RENTEASE

System Architecture Document

Online Property Rental Platform

Project Name	RentEase
Group no.	15
Document Type	System Architecture
Date	28 May 2026
Status	Phase 1 Planning

1. Project Overview

RentEase is a full-stack online property rental platform designed for the Indian market. It enables landlords to list residential properties and tenants to discover, filter, and book them — all within a single responsive web application. The platform's standout feature is an AI-powered rent prediction engine that helps landlords price properties fairly and gives tenants a transparent "Fair Rent Estimate" badge on every listing.

1.1 Problem Statement

The Indian rental market suffers from three core pain points:

- Lack of price transparency — tenants have no baseline to assess whether a listing is overpriced.
- Fragmented discovery — landlords on existing platforms get minimal tools to price competitively.
- Manual, offline booking flows — most platforms lack integrated booking, status tracking, and payments.

RentEase addresses all three by combining a streamlined marketplace with an ML-backed pricing layer.

1.2 Target Users

User Role	Primary Goal	Key Actions
Tenant	Find and book a suitable rental home	Search, filter, wishlist, request booking, pay, review
Landlord	List properties and manage bookings	Add listings, upload photos, accept/reject requests, view dashboard
Admin	Maintain platform quality and compliance	Approve landlords, remove listings, view analytics

2. Technology Stack

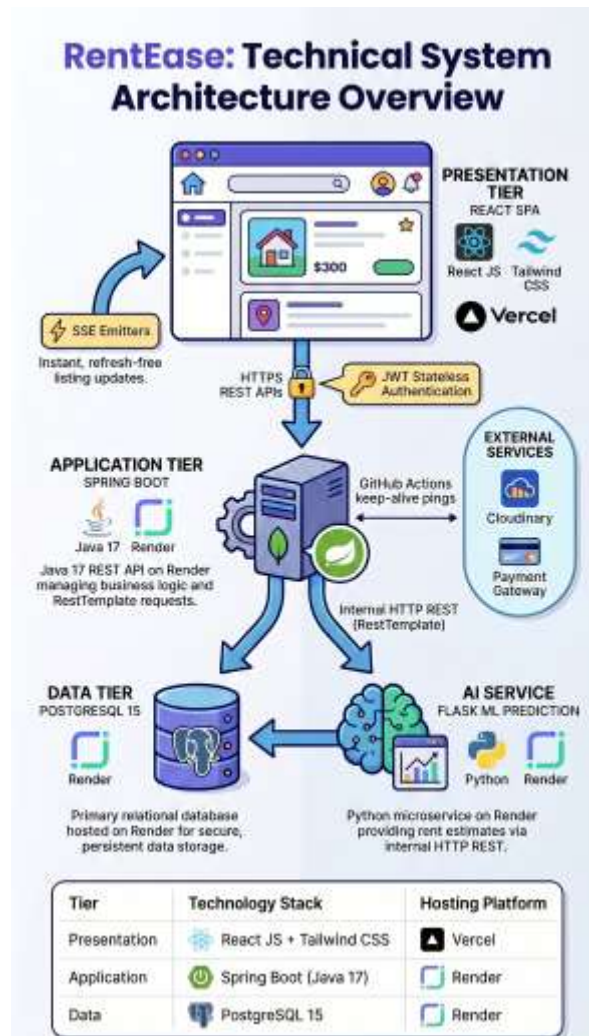
RentEase is built as a three-tier architecture with a separate AI microservice. All components are open-source and deployable on free-tier cloud platforms.

Layer	Technology & Justification
Frontend	React JS + Tailwind CSS — component-based SPA with utility-first responsive styling
Backend	Spring Boot (Java 17) — production-grade REST APIs with battle-tested security
Database	PostgreSQL 15 — relational DB with strong support for geospatial queries (future)
AI / ML Service	Python 3.10 + Flask + Scikit-learn — lightweight ML microservice, decoupled from main backend
Authentication	Spring Security + JWT — stateless, role-based access control (TENANT / LANDLORD / ADMIN)
Payment Gateway	Free open source Payment Gateway
Image Storage	Cloudinary (free tier) — 25GB storage, CDN delivery, transformation APIs
Keep-Alive	GitHub Actions cron (every 14 min) — prevents Render free tier cold starts
Build Tool	Maven — dependency management and CI/CD packaging
Version Control	Git + GitHub — feature-branch workflow with protected main branch

3. System Architecture

3.1 High-Level Architecture

The system is composed of four independently deployable services that communicate over HTTP REST:



3.2 Request Flow

- The React frontend communicates exclusively with the Spring Boot backend via REST APIs.
- Spring Boot handles all business logic, auth, and data persistence in PostgreSQL.
- For rent prediction, Spring Boot calls the Flask ML service as an internal HTTP client using RestTemplate.
- The Flask service loads a pre-trained Random Forest model and returns a rent range in milliseconds.
- Realtime listing updates are pushed to connected tenants via SSE emitters managed in Spring Boot.
- Payments flow through Payment Gateway— order creation on backend, confirmation via signed webhook.

3.3 Authentication & Authorization

All protected endpoints require a valid JWT in the Authorization header. Spring Security filters validate the token and extract the user role before any business logic executes.

Role	Access Scope	Restricted Endpoints
TENANT	Search, wishlist, booking, review, payment	Cannot access landlord or admin routes
LANDLORD	CRUD on own listings, booking management, AI prediction	Cannot modify other landlords' listings
ADMIN	Full platform access, analytics, user management	Superuser — no restrictions

4. Database Schema

PostgreSQL is used as the primary data store. The schema is designed with normalization to support efficient search queries with multiple filter combinations.

Table	Primary Key / Foreign Keys	Purpose
users	id (PK)	Stores all users — tenants, landlords, admins — with role enum
properties	id (PK), landlord_id (FK → users)	Core listing data: city, locality, BHK, sqft, rent, furnished, floor, status
amenities	id (PK)	Master list of amenities (WiFi, parking, gym, etc.)
property_amenities	property_id (FK), amenity_id (FK)	Many-to-many join between properties and amenities
property_images	id (PK), property_id (FK)	Cloudinary URLs for property photos
booking_requests	id (PK), tenant_id (FK), property_id (FK)	Booking lifecycle: PENDING → ACCEPTED / REJECTED
payments	id (PK), booking_id (FK)	Payment Gateway order ID, payment ID, signature, status
reviews	id (PK), tenant_id (FK), property_id (FK)	Rating (1–5) and comment after a completed stay
wishlist	id (PK), tenant_id (FK), property_id (FK)	Saved properties per tenant
rent_predictions	id (PK), property_id (FK)	Cached ML predictions to reduce Flask API calls

4.1 Key Design Decisions

- status column on properties uses an enum: DRAFT, ACTIVE, SUSPENDED, DELETED — soft deletes only.
- booking_requests.status uses enum: PENDING, ACCEPTED, REJECTED, COMPLETED.
- rent_predictions table caches ML results; cache is invalidated when property details change.
- All foreign keys have ON DELETE RESTRICT to prevent orphaned records.

5. REST API Design

5.1 Authentication APIs

Method + Endpoint	Auth Required	Description
POST /api/auth/register	None	Register as TENANT or LANDLORD with name, email, password, role
POST /api/auth/login	None	Returns JWT token on valid credentials
GET /api/auth/me	JWT	Returns authenticated user profile and role

5.2 Property APIs

Method + Endpoint	Auth Required	Description
GET /api/properties	None	Search with query params: city, type, minRent, maxRent, furnished
GET /api/properties/{id}	None	Full property detail with amenities, images, AI estimate
POST /api/properties	LANDLORD	Create new listing
PUT /api/properties/{id}	LANDLORD (owner)	Update listing details
DELETE /api/properties/{id}	LANDLORD (owner)	Soft-delete listing
POST /api/properties/{id}/images	LANDLORD (owner)	Upload photos to Cloudinary
GET /api/listings/stream	None (SSE)	Server-Sent Events stream for realtime new listings

5.3 Booking & Payment APIs

Method + Endpoint	Auth Required	Description
POST /api/bookings	TENANT	Submit booking request with message
GET /api/bookings/tenant	TENANT	View own booking requests and statuses
GET /api/bookings/landlord	LANDLORD	View all incoming booking requests
PUT /api/bookings/{id}/status	LANDLORD	Accept or reject a booking request
POST /api/payments/create-order	TENANT	Create Payment order for booking deposit
POST /api/payments/verify	TENANT	Verify payment signature

5.4 AI Prediction API (Flask)

Method + Endpoint	Called By	Description
POST /predict	Spring Boot internally	Accepts property features, returns {min_rent, max_rent, suggested}
GET /health	GitHub Actions cron	Keep-alive ping endpoint

6. Unique & Differentiating Features

The following features do not currently exist on major Indian rental platforms such as MagicBricks, 99acres, or NoBroker. They are planned across both phases and form the platform's competitive edge.

Feature 1 — AI Fair Rent Badge (Phase 2)

Every listing displays a 'Fair Deal', 'Great Value', or 'Above Market' badge based on ML prediction.

Helps tenants make informed decisions without external research.

Helps landlords set competitive prices — boosting listing conversion rates.

Feature 2 — Realtime Listing Updates via SSE (Phase 1)

When a landlord publishes a new property, connected tenant browsers receive the update instantly

via Server-Sent Events — no page refresh required.

Implemented with Spring Boot SseEmitter; zero external infrastructure needed.

Feature 3 — Rent Negotiation Counter-Offer (Phase 1)

Tenant can propose a counter-price when submitting a booking request.

Landlord can accept the counter, propose their own counter, or reject.

Simple in-app negotiation thread tied to the booking record.

Feature 4 — Move-In Cost Calculator (Phase 1)

Interactive widget on every listing detail page.

Auto-computes: Security Deposit (2–3 months rent) + First Month Rent + Brokerage (if any)

= Total Upfront Cost. Removes the most common tenant uncertainty before applying.

Feature 5 — Roommate-Friendly Listings (Phase 1)

Landlords can flag a listing as 'Flatmate-friendly'.

Tenants can submit a joint booking request as a group (up to 4 co-applicants).

Helps fill larger 3BHK/4BHK properties faster.

Feature 6 — Document Checklist (Phase 1)

Landlords specify required documents (Aadhaar, salary slip, company ID, etc.).

Tenants tick off their prepared documents before submitting a request.

Reduces back-and-forth and accelerates the verification step.

7. Project Phases & Timeline

The project is executed in four phases across.

Phase 0 Day 1	Environment Setup — Git repo, IDE, DB, project scaffolding for all 4 layers
Phase 1 Days 2–18	Core Functionality — Auth, Listings, Booking, Payments, Wishlist, Reviews, Landlord Dashboard, Admin Panel
Phase 2 Days 19–23	AI Rent Prediction — Dataset prep, model training, Flask service, Spring Boot integration, badges
Phase 3 Days 24–28	Polish & Deploy — Responsive UI, testing, Vercel + Render deployment, keep-alive cron

8. Deployment Architecture

All components are hosted on free-tier cloud platforms, making the project fully accessible online at zero cost.

Component	Platform	Notes
React Frontend	Vercel (free hobby plan)	Auto-deploys from GitHub on push to main. Custom domain available.
Spring Boot Backend	Render (free web service)	512MB RAM. Spins down after 15 min idle — mitigated by keep-alive cron.
Flask ML Service	Render (free web service)	Separate service from backend. Gunicorn WSGI server.
PostgreSQL Database	Render (free PostgreSQL)	1GB storage. Connection pooling via HikariCP in Spring Boot.
Images	Cloudinary (free tier)	25GB storage, global CDN, automatic image transformations.
Keep-Alive Cron	GitHub Actions (free)	Pings backend and Flask /health every 14 minutes to prevent cold starts.